

ThunderKittens: Simple, Fast, and *Adorable* AI Kernels

Benjamin F. Spector, Simran Arora, Aaryan Singhal, Daniel Y. Fu, and Christopher Ré

Stanford University

{bfs,simarora,aaryan04,danfu,chrismre}@stanford.edu

October 29, 2024

Abstract

The challenge of mapping AI architectures to GPU hardware is creating a critical bottleneck in AI progress. Despite substantial efforts, hand-written custom kernels fail to meet their theoretical performance thresholds, even on well-established operations like linear attention. The diverse hardware capabilities of GPUs might suggest that we need a wide variety of techniques to achieve high performance. However, our work explores whether a small number of key abstractions can drastically simplify the process. We present THUNDERKITTENS (TK), a framework for writing performant AI kernels while remaining easy to use and maintain. Our abstractions map to the three levels of the GPU hierarchy: (1) at the warp-level, we provide 16x16 matrix tiles as basic data structures and PyTorch-like parallel compute operations over tiles, (2) at the thread-block level, we provide a template for overlapping asynchronous operations across parallel warps, and (3) at the grid-level, we provide support to help hide the block launch and tear-down, and memory costs. We show the value of TK by providing kernels that match or outperform prior kernels for a range of AI operations. We match CuBLAS and FlashAttention-3 on GEMM and attention inference performance and outperform the strongest baselines by 10 – 40% on attention backwards, 8× on state space models, and 14× on linear attention.

1 Introduction

AI is bottlenecked by the problem of efficiently mapping AI architectures onto accelerated GPU hardware. There has been a Cambrian explosion of ML architectures [21, 25]; however, the performance of these architectures remains substantially below their theoretical potential, despite substantial effort to develop kernels, or GPU implementations. Notably, kernel support has been poor even for softmax attention, which is used throughout industry. FlashAttention-2 [12] suffered a 47% performance degradation when translated to the H100 GPU, and it took over two years from the release of the H100 to develop FlashAttention-3 [37].

We are inspired by several approaches to supporting the development of AI kernels. Ideally, we would have a framework that supports high performance for a breadth of primitives, while being easy to use, learn from, and maintain. High performance C++ embedded libraries like NVIDIA CUTLASS/CuTe [29] contain a myriad of nested templates, while compiler based approaches like Triton [39] provide users with simpler interfaces, but fewer optimizations. We ask how broad and fast we can go by choosing a small and opinionated set of abstractions.

The main vector of growth for accelerated compute is in specialized matrix multiply units. On the NVIDIA A100 and NVIDIA H100 GPUs, BF16 tensor cores represent 16× the FLOPs available relative to general-purpose BF16 / FP32 compute. Consequently, any high performance framework must prioritize keeping tensor cores at high utilization whenever possible. However, all kernels have other operations too (like memory loads or the softmax in attention), and it is crucial to minimize the overhead of non-tensor core operations. This proposition is at the heart of our approach.

To understand the complexities and opportunities in building a simple, yet high performance framework, we examine a simplified model of GPU parallelism, further detailed in section 2.¹

¹We discuss NVIDIA, but the parallelism types hold across architectures, including AMD and Apple GPUs.

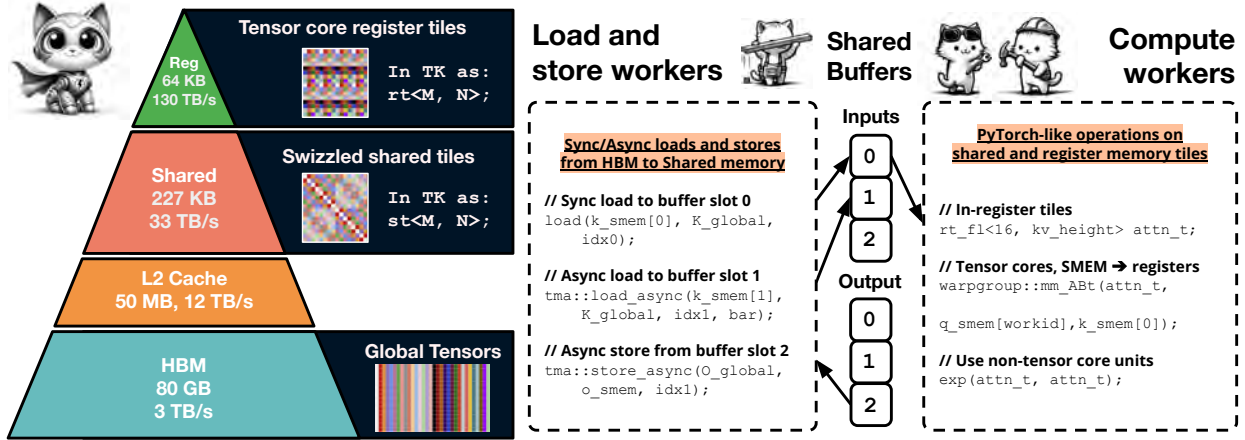


Figure 1: THUNDERKITTENS explores whether a small set of abstractions can support efficient and simple AI kernels. Inspired by PyTorch, our abstractions include tiles with managed layouts and operations over tiles. We provide a general program template for coordinating asynchronous parallel workers – e.g., workers that load from and store data to HBM, while other workers perform computation in fast memory.

- Warp-level parallelism:** Modern GPUs consist of tens of thousands of hardware threads which execute in parallel. Threads are organized into small groups, “warps”, which execute instructions together. Memory layouts determine how the logical data elements are mapped to physical thread ownership. If multiple threads try to access the same region (“bank”) of memory, this can create expensive serializations between the threads (called “bank conflicts”).
- Block-level parallelism:** Warps are grouped into “blocks” of threads, which can quickly share data. Warps execute their instructions on physical execution units, and having more warps in a block (called *occupancy*) can help run more instructions at the same time, reducing runtime. For example, one warp can run tensor cores for matmul, while another uses the ALU for max.
- Grid-level parallelism.** GPUs run many blocks of threads at once, which communicate through large but slow global memory (HBM). An on-chip shared L2 cache helps reduce memory latencies and increase bandwidth if thread blocks reuse the same data. Thread blocks also face setup and tear-down latency overheads, called “pipeline bubbles”

Despite the apparent need for a myriad of techniques to leverage all these hardware capabilities, our central technical finding is that indeed, for many AI kernels, a small number of key abstractions exist that can simplify the process of writing high-performance kernels. Our exploration led us to develop THUNDERKITTENS (TK), an AI kernel framework built around three key principles:

- Tile data structures with managed layouts:** Our interface is inspired by familiar ML frameworks like PyTorch and NumPy [31], as highlighted in Figure 2. At the warp level, we use a 16×16 matrix tile as our basic data structure, maximizing compatibility with and encouraging the use of tensor cores. TK automatically picks the optimal memory layouts for the tiles to minimize bank conflicts while remaining compatible with specialized hardware instructions, avoiding user effort. We provide a set of parallel compute primitives over tiles, based on the suite of operations in PyTorch (e.g., pointwise multiply, mma, exp, and cumsum over tiles).
- Program template for asynchronous work:** At the block level, TK provides a general kernel template for coordinating asynchronous execution across warps in a thread block, built on the producer-consumer paradigm [16]. The developer’s effort reduces to populating a few boilerplate functions within this model, using our PyTorch-like operands, and the template internally hides latencies through memory pipelines and synchronization primitives (Figure 1).
- Grid scheduling for pipelining thread-blocks.** At the grid level, we show TK can help developers reduce pipeline bubbles and improve L2 cache hit rates. Our template supports a *persistent grid*, where we overlap memory loads across thread block boundaries.

PyTorch attention:

```

1  # imports
2  import torch
3  import torch.nn.functional as F
4
5  # compute Q@K.T
6  att = torch.matmul(
7      q, k.transpose(2, 3))
8
9  # compute softmax
10 att = F.softmax(
11     att, dim=-1,
12     dtype=torch.float32)
13
14 # convert back to bf16
15 att = att.to(q.dtype)
16
17 # mma att@V
18 attn_output = torch.matmul(att, v)

```

THUNDERKITTENS attention:

```

1  // imports
2  using namespace kittens;
3  rt_bf<16, 64> k_reg, v_reg;
4  // load k from shared memory to register
5  load(k_reg, k_smem[subtile]);
6  // compute Q@K.T
7  zero(att);
8  mma_ABt(att, q_reg, k_reg, att);
9  // compute softmax
10 sub_row(att, att, max_vec);
11 exp(att, att);
12 div_row(att, att, norm_vec);
13 // convert to bf16 for mma_AB
14 copy(att_mma, att);
15 // load v from shared memory to register
16 load(v_reg, v_smem[subtile]);
17 auto &v_reg_col = swap_layout_inplace(v_reg);
18 // mma att@V onto o_reg
19 mma_AB(o_reg, att_mma, v_reg_col, o_reg);

```

Figure 2: Attention implemented in TK using familiar PyTorch-like operations on tiles.

We highlight the value of these abstractions for developers in two ways:

- Through our exploration, we identify a few fundamental tradeoffs between achieving different types of parallelism including in setting the tile layouts (warp-level), occupancy (block level), and block launch order (grid level). Through our ablation studies (Section 3), we show how the simplified interface in TK gives users the control to navigate the tradeoffs.
- We validate the TK abstractions by providing kernels that match or outperform prior kernels for a range of AI operations. We match CuBLAS GEMMs and FlashAttention-3 attention inference, and outperform the strongest baselines by 10 – 40% on attention backwards, up to 8× on state space models, and up to 14× on linear attention. These kernels are written by a small academic team, including by undergraduates with no prior CUDA experience.

Our contributions are (1) showing a small and opinionated set of abstractions in TK that goes surprisingly far for writing simple and performant kernels; and (2) providing a collection of performant AI kernels. TK kernels are in production at ML inference providers and high-frequency trading firms alike. We hope that TK and its insights help improve the accessibility of AI kernels.

2 GPU fundamentals

GPU tasks are divided into small programs called *kernels*. A kernel loads data from high bandwidth memory (HBM), performs work on it, and writes the outputs back to HBM before concluding. Before we explain THUNDERKITTENS’s abstractions, we provide background on GPU parallelism including *warp*, *block* and *grid-level* parallelism. We follow NVIDIA’s terminology and focus on the H100 SXM GPU, though the principles apply across GPU vendors and generations.

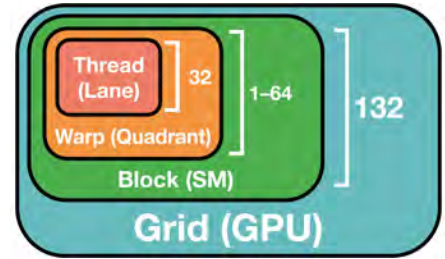


Figure 3: The software (and physical) GPU hierarchy.

2.1 GPU hierarchy

The GPU software hierarchy closely follows its physical hardware hierarchy (Figure 3). Here, we illustrate several of its most important components and aspects.

1. **Warps** consist of groups of 32 nearby *threads* that operate on data in small but fast *register* memory. These instructions run on various *physical execution units*, which are specialized for different compute operations (below) and different threads can occupy different units simultaneously:

- (a) **Load and store units**, to bring data into and out of registers. Advanced GPUs have also introduced **dedicated hardware** acceleration for bulk loads and stores.
 - (b) General purpose compute pipelines, such as **ALU** for max, min, **FMA** for multiplies and adds, and **XU** for complex operations like exp. Throughput differs across the pipelines.
 - (c) Accelerated **matrix multiply hardware (tensor cores)**, which have most of the GPU compute.
2. **Thread blocks** are groups of warps which together execute a kernel on a physical core, called a *streaming multiprocessor* (SM). Although each SM has just four physical execution units, up to 64 software warps can simultaneously run on it (called “occupancy”). These collocated warps often contend on hardware resources: registers, shared memory, issue slots, and compute pipelines, but together they can help keep many work streams running at the same time within each execution unit. Warps *synchronize* at barriers, during which they cannot issue new work.

Importantly, warps within the same block can quickly communicate through special *shared memory* (SMEM, 227 KB, 33 TB/s). To improve bandwidth, SMEM is grouped into 32 physical “banks” of memory, which can serve memory simultaneously. However, if different threads try to access the same bank at the same time (called a *bank conflict*), their accesses must be serialized, which both increases access latencies and reduces available bandwidth. Hopper has limit of 255 registers per thread and attempts to request more, results in *spills* to the L1 cache. SMEM can be reallocated as an *L1 cache* for fast access to frequently used memory like spilled registers.

3. **Grids** of multiple thread blocks are launched to run the kernel. The H100 SXM GPU has 132 physical SM’s which can run thread blocks at the same time. Although SM’s are capable of collocating multiple thread blocks, most AI kernels can achieve high performance by simply collocating more warps within a single thread block (increasing the occupancy).

Thread blocks on the same GPU share common memory resources: large but slow high-bandwidth memory (80 GB, 3 TB/s), which has both the greatest latency and least bandwidth of all GPU memory, and a smaller but faster L2 cache (50 MB, 12 TB/s).

There are overheads to scheduling blocks. First, the block launch incurs *setup* costs and although this cost must be paid at least once at the initial kernel launch, kernels that continuously launch many large blocks can incur further costs. Second, there are *tail effect* costs if the grid is sized poorly. If a kernel of 133 blocks is executed on an H100 with 132 physical SMs, the kernel would require two waves to execute, the first with full efficiency, and the second with < 1% efficiency.

2.2 Cost model

Beyond reducing the total amount of work, the other key way to reduce execution time is to overlap multiple kinds of work at once. Summarizing the above components, we provide a simplified cost model for GPU parallelism. We break down the overall kernel execution time C_{Overall} as:

$$C_{\text{Overall}} = \max \left(\underbrace{C_{\text{HBM}}, C_{\text{L2}}, C_{\text{L1}}, C_{\text{Shared}}}_{\text{Memory}}, \underbrace{C_{\text{Tensor}}, C_{\text{ALU}}, C_{\text{FMA}}, C_{\text{XU}}}_{\text{Compute}} \right) + \underbrace{C_{\text{Setup}} + C_{\text{Sync}}}_{\text{Overhead}}$$

where memory costs are a combination of the latency and bandwidth, and compute costs are a combination of latency and throughput.

This model represents the *ideal case* of perfect overlapping between memory, compute, and tensor core costs. A kernel’s actual performance will lie between the max and the sum of these components, depending on the workload properties (*i.e.*, some operations are inherently sequential), as well as the efficiency of its implementation. Nonetheless, our explorations will be guided by trying to (1) reduce these individual costs, and (2) improve their collective overlapping.

2.3 GPU programming frameworks

We are inspired by a number of related efforts to simplify the development of AI kernels, such as NVIDIA CUTLASS/CuTe [29] and Triton [39].

CUTLASS’s myriad of nested CUDA templates helps power highly optimized AI kernels [8, 9, 37] and fundamentally, the same kernels are expressible in TK and CUTLASS, since both are *embedded* libraries, giving users the full power of C++. We take a complementary approach by being rather opinionated about the abstractions. We ask: (1) *How far can we get with a small of templates?* and (2) *Does concision sacrifice performance?* An appealing outcome is improved accessibility to AI researchers, since it can be challenging to fully leverage the capabilities of CUTLASS [9]. We find that even industrially popular kernels written in CUTLASS, like FlashAttention-3, struggle from preventable issues like bank conflicts. We seek abstractions that manage such issues for users. Most recent AI architectures use high level compilers instead [13, 20, 44].

Triton, PyTorch [31], TVM [10], TensorFlow XLA [1], and others approach the problem from a compiler perspective. The frameworks are not C++ embedded, so it can be challenging to use unsupported specialized hardware instructions. It can also be difficult to manage asynchronous execution and register usage in high level frameworks. We explore avenues that retain the simple, PyTorch-like feel and enable high performance in the next section. An extended discussion of related work is in Appendix A.

3 ThunderKittens

We present THUNDERKITTENS (TK), a framework designed to simplify the development of high-performance AI kernels while leveraging the full capabilities of modern GPUs. This section (1) introduces our key programming abstractions and (2) shows how they can help developers navigate the tradeoffs between different types of parallelism. Section 3.1 focuses on warp level, Section 3.2 on thread block level, and Section 3.3 on grid level parallelism.

As running examples in this section, we show how TK helps optimize attention [41] and GEMM kernels. Section 4 demonstrates how the principles yield performant kernels for a breadth of AI operations (*e.g.*, attention variants, convolution, SSM, rotary).

3.1 Warp parallelism with familiar data structures and operations

At its core, THUNDERKITTENS is built on two fundamental abstractions – **tile data structures** at each level of the memory hierarchy and **bulk operands on tiles** akin to the familiar suite of operations in PyTorch and NumPy. We first define the abstractions, and then show they can help developers navigate tradeoffs between the tile *sizes* and efficiency.

Programming abstractions TK is heavily inspired by PyTorch and NumPy, given their familiarity to ML audiences [31]. We provide a concise set of parallel compute operations, based on the suite of operations in PyTorch (*e.g.*, in Figure 2). The operations are executed by a “worker” abstraction, or a warp or warpgroup (4 warps) of threads that collaboratively own and operate on a piece of data. TK uses a 16×16 matrix tile as its basic data structure, designed to maximize compatibility with tensor cores. We provide tiles for each level of the memory hierarchy:

1. Register tiles and vectors, which are templated by type, shape, and layout. In Figure 2 we initialize a bfloat16 type tile with a column-major layout, height 16, width 64.
2. Shared tiles and vectors, which are templated by type and shape.
3. Global layout descriptors: We set up HBM loads and stores as indexing into 4D tensors (similar to {batch, head, length, and embed} in PyTorch). Dimensions can be known at compile-time or runtime. Compile-time dimensions can be stored in the instruction cache, saving registers.

An advantage of these tile-based abstractions is that they enable TK to statically check layouts and operations, which is important because GPU kernels are often difficult to debug. For example, an in-register tensor core multiply `mma_AB` requires *A* to be in a row-major layout, and *B* to be in a column-major layout, and TK can raise compile-time errors if these conditions are not met.

Choosing a memory layout Layouts specify how logical data elements are mapped to physical thread ownership. Different execution units, tile sizes and types, and hardware-accelerated instructions benefit from different layouts. A poor choice of layout can lead to bank conflicts (`CShared`, Section 2). Our goals are:

- We want our register tiles (the fastest GPU memory) to by-default keep memory in the layouts required by tensor core units (the fastest GPU compute units). Shown in Figure 1 (Left), where each color represents

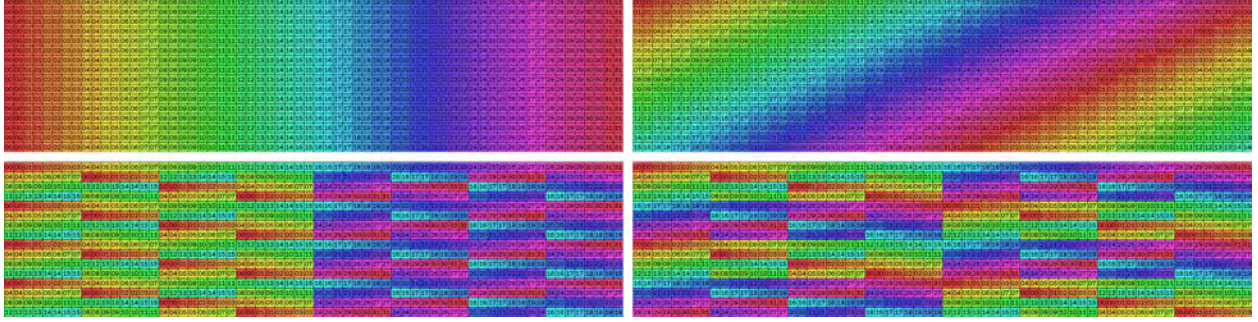


Figure 4: Shared memory bank layouts, illustrated for a 16x64 16-bit tile. **Top left:** A naive, row-major layout. Although loading rows is efficient, loading into a tensor core layout suffers 8-way bank conflicts. **Top right:** A padded layout, which has no bank conflicts but consumes additional memory and has poor hardware support. **Bottom:** Two of TK’s three chosen layouts, with compile-time selection based on width. (Bank conflicts are unavoidable for some tile sizes while maintaining good hardware support.) These layouts have 2-way and no bank conflicts, respectively.

a different thread’s ownership over the data elements, the tensor formats are rather difficult to use and reason about, highlighted in our discussion of naive layouts in Figure 4.

- We want to support the use of hardware-accelerated instructions (*e.g.*, asynchronous matrix multiply and bulk copy instructions), which also require specific shared memory layouts.

In TK, we simplify the search space to 3 layouts – strided at 32, 64, and 128 byte intervals – and automatically give shared tiles with the largest layout that the tile size supports to minimize bank conflicts. Highlighted in Section 4.2, even optimized FlashAttention-3 kernel written using CUTLASS and CuTe templates suffer from bank conflicts, hurting performance. Our approach helps minimize conflicts.

3.2 Block parallelism with a generalized asynchronous template

THUNDERKITTENS helps developers reduce overheads by coordinating how workers in a thread block asynchronously overlap execution. Though the GPU hierarchy might suggest that we need a wide variety of techniques, we propose a *single* concise template that we find enables high performance on a surprisingly broad range of AI workloads. We first define the template, which has four steps – load-compute-store-finish (LCSF for short) – and builds on the classical producer-consumer paradigm [7, 16]. We then show how the LCSF template can help developers navigate the tradeoffs between occupancy and efficiency.

Load function:

```

1 if(warpgroup::warpid() == 0) {
2   tma::expect(inputs_arrived,
3     block.k, block.v);
4   tma::load_async(
5     block.k, globals.k,
6     {batch, head, iter, 0},
7     inputs_arrived);
8   tma::load_async(
9     block.v, globals.v,
10    {batch, head, iter, 0},
11    inputs_arrived);
12 }
13 else arrive(inputs_arrived);

```

Compute function:

```

1 warpgroup::mm_ABt(att, scratch.q[state.id], block.k);
2 warpgroup::mma_async_wait();
3
4 // softmax (simplified)
5 sub_row(att, att, max_vec);
6 exp(att, att);
7 div_row(att, att, norm_vec);
8
9 copy(att_bf16, att);
10
11 warpgroup::mma_AB(state.o, att_bf16, block.v);
12 warpgroup::mma_async_wait();
13 arrive(inputs_finished);

```

Figure 5: An attention kernel within the LCSF template. The left shows the functionality for workers focused on managing HBM to SRAM memory movement, and the right shows the functionality for parallel compute workers that operate in fast memory, registers and SRAM.

Programming abstractions As discussed in Section 2, the general pattern of an AI kernel is to load tiles of large tensors from HBM to SRAM, perform computation in fast memory, store the result for the tile back to HBM, and repeat this for the next tiles. To use the LCSF template, the developer writes four functions:

1. **Load function:** The load function specifies the data that load workers should load from HBM to shared memory, and when to signal to compute workers that this memory is ready for use.
2. **Compute function:** This function specifies the kernel instructions that compute workers should execute, using the tile data structure and operation primitives from Section 3.1.
3. **Store function:** The store function specifies what data needs to be stored to HBM by store workers.
4. **Finish function:** At the end of the kernel, the workers store any final state and exit.

TK provides abstractions to help the developer manage worker overlapping and synchronization.

1. **Multi-stage buffer:** The template maintains N -stage pipelined buffers in shared memory, which are used for loads and stores from HBM. Load/store workers add/remove tiles of data from the buffers, based on the status of compute workers. With a single stage, load workers would need to wait for all compute workers to finish executing before replacing the input tile. A 2-stage buffer can hide the HBM load (store) latency since the next tile can asynchronously load, while the compute workers execute on the current tile. Deep buffers can reduce the amount of synchronization required across compute workers, allowing them to operate on different tiles concurrently.

TK lets the user set a single number to specify the number of stages, and manages the setup and use of these buffers for the user. We demonstrate this in Section 3.2, where we vary the number of stages $N \in \{1, 2, 3, 4\}$ for our GEMM kernel.

2. **Synchronization barriers:** Load/store workers need to alert compute workers when new memory is written to the input buffer. Compute workers need to alert load/store workers when tiles are written to the output buffer, or when input tiles can be evicted from the input buffer. Within the TK template, we provide an **arrive** function for workers to signal that they have finished their stage.

3. **Asynchronous I/O:** We wrap synchronous and asynchronous load and store instructions, including `cp.async` and TMA, in the same interface. We automate tensor map descriptor creation for TMA hardware-accelerated address generation for our global layout descriptors (g1).

Tradeoffs between occupancy and efficiency

TK parametrizes the number of load/store and compute workers (or occupancy) providing a simple way for developers tune their kernels. As discussed in Section 2, higher occupancy increases overlapping, but creates contention over limited hardware resources (e.g., registers). With fewer registers, workers need to operate on smaller tiles of data, resulting in more instruction issues, SRAM to register I/O, and potentially higher synchronization costs due to the increased data partitioning across workers.

Figure 6 shows the occupancy tradeoffs for attention kernels. We consider (1) a simple kernel that only uses warp level parallelism (Listing 2) and (2) a kernel written in the LCSF template (Listing 5). With both kernels, performance increases until resource contention dominates. Second, we observe that LCSF expands the Pareto frontier beyond the warp-level parallel kernel as we vary occupancy.

M = N = K	Stages	TFLOPS
4096	1	260
4096	2	484
4096	3	683
4096	4	760

Table 1: **Pipeline buffer stages** We measure efficiency in TFLOPS for our GEMM kernels as we vary the number of pipeline buffer stages in the TK template.

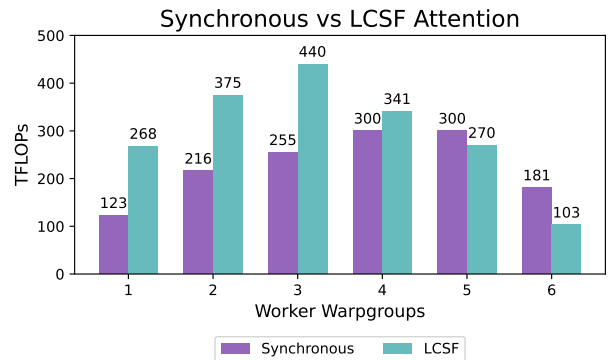


Figure 6: **Occupancy tradeoff:** (Left) Attention TFLOPs as a function of occupancy, benchmarked with head dimension 64 and a sequence length of 4096. We compare a basic synchronous and LCSF kernel.

Matrix Multiply (M=N=K=16384)			Attention Forward (D=128)		
Block Order	HBM GB/s	TFLOPS	Block Order	HBM GB/s	TFLOPS
{8, N, M/8}	982	805	{N, H, B}	213	600
{N, M}	3,070	392	{B, H, N}	2,390	494

Table 3: **L2 reuse** We vary the block orders and measure both consumed bandwidth from HBM (GB/s) and efficiency (TFLOPS). For attention, we consider an optimized kernel, with an internal tiling of 8 rows of blocks, versus a naive kernel that schedules blocks in row-major order. For attention, we compare block order (1) sequence length N , heads H , and outermost batch B vs. (2) innermost B , H , then outermost N . Different block orders have significant performance implications.

We find the general LCSF template to be effective across a range of AI workloads. We keep the template lightweight and simple by making opinionated design choices. However, we don’t want TK to get in the way of achieving peak GPU performance – in contrast to frameworks like Triton, TK is *embedded*, meaning developers can use the full power of C++ to extend the library as warranted.

3.3 Grid parallelism with block launch scheduling

Next, at the grid level, we explore how to coordinate thread block launches. TK’s template does not explicitly choose grid structures for the user, however we provide a tradeoffs study of two key opportunities: reducing the setup and tear-down costs for each thread block (C_{Setup}), and encouraging memory reuse between thread blocks to avoid slow HBM accesses (C_{HBM}).

Block launch costs We first explore the use of a *persistent grid*, where we launch thread blocks on the full set of 132 SMs upfront, and simply load the next chunk of work for the kernel within the existing block, instead of launching a new block. We also explore the idea of having the block load shared memory into the input stage of our template’s memory buffer to prepare for the next chunk of work, while the thread block runs the finish stage for the prior chunk of work. Table 2 shows the benefit of the optimizations for our GEMM kernels.

M=N	K	TK-No	TK-Yes	CuBLAS
4096	64	93	108	69
4096	128	161	184	133
4096	256	271	309	242
4096	512	414	450	407
4096	1024	565	600	633

Table 2: **Persistent block launch** TFLOPS for TK GEMM kernels with (yes) persistent and without (no) persistent launch as we vary matrix dimension K .

L2 reuse and block launch order Recall that thread blocks need to communicate via HBM. As introduced in Section 2, when thread blocks reuse memory, the data is often available in L2 cache, which is significantly faster than HBM. However, cache eviction means that these reuse qualities depend on the order in which blocks get launched. For our attention and GEMM kernels, we measure efficiency as we vary block order, summarized in Table 3. Block order substantially affects L2 reuse (measured through HBM bandwidth), which in turn can control kernel performance.

4 Experiments

In experiments, we validate that THUNDERKITTENS speeds up a broad range of ML primitives. We compare to well-optimized kernels from prior work, written in alternate frameworks such as CutLass, CuBLAS, general CUDA, and Triton. We compare our kernels for the “workhorse” operations in AI, GEMM and attention, as well as kernels for emerging AI architectures, such as linear attention and state space models (Section 4.1). We profile the kernels to understand TK’s role in achieving high performance in Section 4.2. Kernel listings, in the TK template, are in Appendix B.

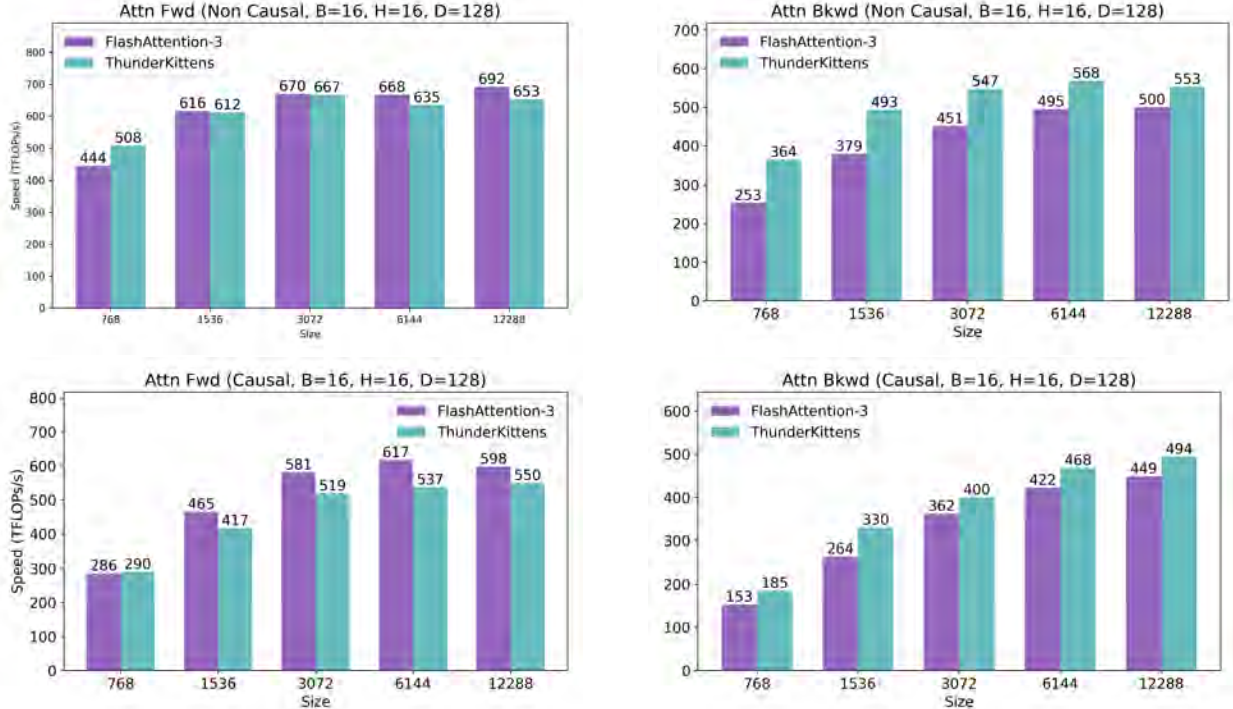


Figure 8: Attention causal and non causal inference and backwards pass efficiency.

4.1 TK enables simple and performant AI kernels

This section shows a suite of kernels that we develop in the TK framework. We benchmark the kernels on NVIDIA H100 80GB SXM GPUs using CUDA 12.6 and report the average TFLOPS.

Workhorse kernels for AI Industry teams and researchers have made significant investments into optimizing GEMMs and attention over the past several years [8, 15, 30, 37, inter alia.], two workhorse operations that power the Transformer architecture [41]. Despite the investment, TK kernels written entirely in the TK abstractions and LCSF template can match or outperform the strongest baselines:

- **GEMM** We compare to the strongest available baseline: CuBLAS for GEMMs [30]. We show a single matrix multiply kernel, with just 40 lines of device code, can compete with CuBLAS
- **Attention.** We support multiple variants of attention: causal, non-causal, and grouped query attention [3] at head dimensions 64 and 128. We compare to the strongest available baseline, which is concurrent to our work: FlashAttention-3 (FA3) [37]. TK competes with FA3 across sequence lengths on the non-causal forwards pass, and outperforms FA3 on the causal and non-causal backwards pass by over 40% at short sequences and 10% at longer sequences.

We find that TK makes it easy to use the GPU effectively by simplifying the choice of memory layouts, exploration of grid patterns for L2 reuse, and selection of occupancy and pipeline depth. The baseline kernels successfully use specialized H100 instructions and manage memory. However, the existing kernels are relatively complex: FlashAttention-3 proposes a “ping-pong scheduler” for workers, and the CuBLAS library is >600MB in CUDA 12.6 (Appendix A), containing many tuned GEMM variants and logic to select the best option at runtime [36]. With TK, we remove the ping-pong and maintain FA3-level efficiency, and we compete

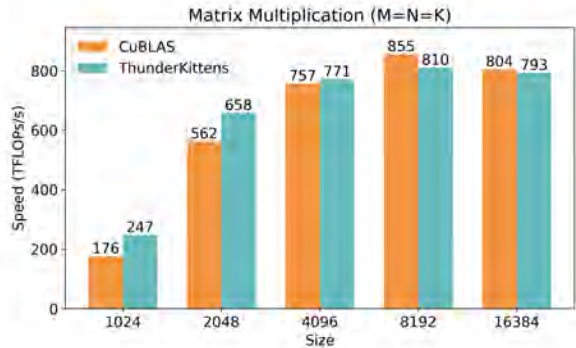


Figure 7: GEMM kernel from CuBLAS and TK.

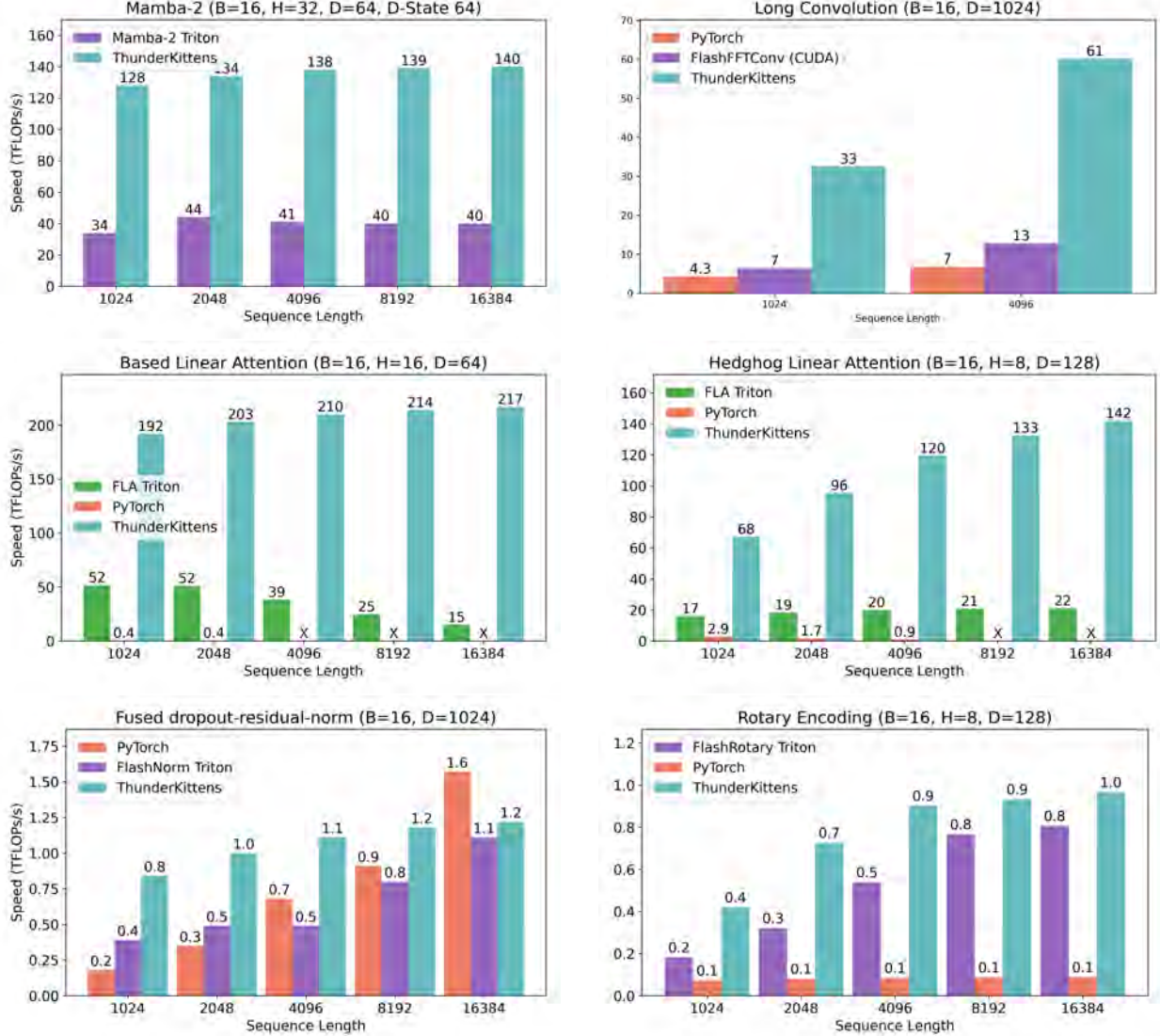


Figure 9: ThunderKittens kernels are performant across a wide range of kernels. We benchmark the kernels on an 80GB NVIDIA H100 GPU and report TFLOPs.

with CuBLAS on the demonstrated matrix sizes, using a single GEMM kernel (entirely in Appendix B.1).

Kernels for emerging AI architectures In addition to supporting peak performance on popular operations like GEMMs and attention, TK is also designed for to be extensible to emerging AI workloads. We release a family of kernels across newer machine learning primitives, including linear attention [27], FFT convolutions [11], and state space models [22].

- **Linear attention** We optimize two different classes of linear attention architectures, polynomial-based feature maps as in [4, 5, 26, 28] and learned feature maps as in [45, 46]. In Figure 9, we compare to the strongest available baselines: the popular Flash Linear Attention (FLA) CUDA kernels [44], which are written in Triton. We show TK outperforms FLA’s polynomial-based linear attention by 14 $\times$. TK outperforms FLA’s learned map linear attention by 6.5 $\times$.
- **State space models** The long convolution, implemented with Fourier transforms using the convolution theorem, is the key primitive in popular state space modeling architectures such as S4, H3, and Hyena [2, 19, 22, 23, 33, inter alia.]. In Figure 9, we compare to the strongest available baseline: the FlashFFTConv CUDA kernels in Fu et al. [20] and show TK outperforms the prior work by 4.7 $\times$ at sequence length 4096 and 7.9 $\times$ at 1024. TK outperforms PyTorch’s FFT operations by up to 8.7 $\times$.

Impl.	Occupancy utilizations (%)		HBM		Shared
	Tensor core	Issue slots	TPS (GB/s)	Stalls (Cycles)	Stalls (Cycles)
FA3 Bkwd	61.2	25.1	328	1.83	0.92
TK Bkwd	58.2	34.8	490	1.63	0.14
FlashFFT	13.4	25.5	14.8	2.5	1.6
TK	54.8	40.0	31.4	0.6	0.3

Table 4: Profiles for 1) attention backwards pass kernels from FlashAttention-3 [37] vs. TK and 2) long convolution kernels from FlashFFTConv [20] vs. TK, obtained using NVIDIA NSight Compute.

We also optimize the recent Mamba-2 state space model [13]. We provide a TK kernel that outperforms the Triton kernels in prior work Dao & Gu [13] by $> 3\times$ (Figure 9). This gap is primarily due to the ease of fusing complex operations in TK.

The abstractions in TK – including specialized instructions such as TMA and WGMMA, and register tiles to manage register memory effectively – contribute to these large improvements. The baseline kernels do not use these GPU features.

TK’s programming model is extensible. We develop kernels for common memory AI operations – fused dropout-residual-layernorm [6], and rotary [38] – and show TK is effective. We compare to popular Triton kernels for these operations.²

4.2 Comparing kernel implementations

To further compare TK and baseline kernels, we profile the kernels using NVIDIA’s NSight Compute (NCU) tool.³ In Table 4, we give NCU profiles for both the emerging long convolution primitive and the well-optimized attention backwards pass, comparing to the strongest respective baselines.

- **Long convolution** We profile FlashFFTConv (FC) and TK long convolution kernels at $B, D, N = 16, 1024, 4096$ in NCU. We find TK helps both with overlapping the workers (indicated by higher issue slots and fewer memory stalls) and in tensor core utilization ($4.1\times$ increase). This is enabled by our TK template, and use of TK warpgroup operations (which saves registers and establishes a SMEM to register memory pipeline through warpgroup matrix-multiply-add (WGMMA) operations).
- **Attention backwards** We consider FA3 and TK at $B, H, N, D = 16, 16, 3072, 128$. The methods match in tensor core utilization, but TK gives higher issue slot utilization, suggesting the occupancy may be better-tuned. In HBM costs, TK gives higher memory throughput and correspondingly incurs 10% fewer stalled cycles on HBM waits. For shared memory, TK incurs 85% fewer stalled cycles – we find TK has *no bank conflicts*, but NVIDIA’s NCU profiler reports up to 9.6-way bank conflicts in FA-3.

The kernel profiles highlight the difficulty of simultaneously managing each type of GPU parallelism, and we hope TK can help reduce the effort. We provide example TK kernels listings in Appendix B.

5 Conclusion

Given the challenge of mapping AI architectures to GPU hardware, our work asks how far we can get with a few easy to use GPU programming abstractions. In THUNDERKITTENS, we give an abstraction for each level of the GPU hierarchy: tiles with managed layouts at the worker level and a asynchronous execution LCSF template at the thread block level. We highlight options and tradeoffs for persistent block launches and L2 reuse at the grid level. The natural question is whether we sacrifice anything in performance when we write kernels with so few abstractions. We implement a breadth of AI kernels in TK and excitingly find that our abstractions are both general and consistently meet or exceed state-of-the-art. We are optimistic about the potential for simple and accessible ways of programming AI hardware.

²Reference triton kernels are from <https://github.com/Dao-AILab/flash-attention>.

³<https://developer.nvidia.com/nsight-compute>

To support future development, our framework and kernels are open sourced at: <https://github.com/HazyResearch/ThunderKittens>.

6 Acknowledgements

We are grateful to Together.ai for making this work possible. We thank Arjun Parthasarthy for assistance in developing the complex tile support and FlashFFT kernels. We thank Mayee Chen, Tri Dao, Kawin Ethayarajh, Sabri Eyuboglu, Neel Guha, David Hou, Jordan Juravsky, Hermann Kumbong, Jerry Liu, Avner May, Quinn McIntyre, Jon Saad-Falcon, Vijay Thakkar, Albert Tseng, Michael Zhang for helpful feedback and discussions during this work. We gratefully acknowledge the support of NIH under No. U54EB020405 (Mobilize), NSF under Nos. CCF2247015 (Hardware-Aware), CCF1763315 (Beyond Sparsity), CCF1563078 (Volume to Velocity), and 1937301 (RTML); US DEVCOM ARL under Nos. W911NF-23-2-0184 (Long-context) and W911NF-21-2-0251 (Interactive Human-AI Teaming); ONR under Nos. N000142312633 (Deep Signal Processing); Stanford HAI under No. 247183; NXP, Xilinx, LETI-CEA, Intel, IBM, Microsoft, NEC, Toshiba, TSMC, ARM, Hitachi, BASF, Accenture, Ericsson, Qualcomm, Analog Devices, Google Cloud, Salesforce, Total, the HAI-GCP Cloud Credits for Research program, the Stanford Data Science Initiative (SDSI), BFS is supported by a Hertz Fellowship, SA is supported by a SGF Fellowship, and members of the Stanford DAWN project: Meta, Google, and VMWare. The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes notwithstanding any copyright notation thereon. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views, policies, or endorsements, either expressed or implied, of NIH, ONR, or the U.S. Government.

References

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. Tensorflow: Large-scale machine learning on heterogeneous distributed systems. *arXiv preprint arXiv:1603.04467*, 2016.
- [2] Naman Agarwal, Daniel Suo, Xinyi Chen, and Elad Hazan. Spectral state space models, 2024. URL <https://arxiv.org/abs/2312.06837>.
- [3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghani. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *The 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [4] Yaroslav Aksenov, Nikita Balagansky, Sofia Maria Lo Cicero Vaina, Boris Shaposhnikov, Alexey Gorbatskiy, and Daniil Gavrilov. Linear transformers with learnable kernel functions are better in-context models. 2024.
- [5] Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalina, Silas Alberti, Dylan Zinsley, James Zou, Atri Rudra, and Christopher Ré. Simple linear attention language models balance the recall-throughput tradeoff. *International Conference on Machine Learning*, 2024.
- [6] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [7] Michael Bauer, Henry Cook, and Brucek Khailany. Cudadma: Optimizing gpu memory bandwidth via warp specialization. *SC '11: Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, 2011.
- [8] Ganesh Bikshandi and Jay Shah. A case study in cuda kernel fusion: Implementing flashattention-2 on nvidia hopper architecture using the cutlass library, 2023. URL <https://research.colfax-intl.com/wp-content/uploads/2023/12/colfax-flashattention.pdf>.

- [9] Ganesh Bikshandi and Jay Shah. Developing cuda kernels for accelerated matrix multiplication on nvidia hopper architecture using the cutlass library, 2023. URL <https://research.colfax-intl.com/wp-content/uploads/2023/12/colfax-gemm-kernels-hopper.pdf>.
- [10] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pp. 578–594, 2018.
- [11] James W Cooley and John W Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of computation*, 19(90):297–301, 1965.
- [12] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *International Conference on Learning Representations*, 2024.
- [13] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *International Conference on Machine Learning (ICML)*, 2024.
- [14] Tri Dao, Beidi Chen, Nimit Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Ré. Monarch: Expressive structured matrices for efficient and accurate training, 2022. URL <https://arxiv.org/abs/2204.00595>.
- [15] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.
- [16] E. W. Dijkstra. Co-operating sequential processes. In F. Genuys (ed.), *Programming Languages*. NATO Advanced Study Institute, 1968. Lectures given at a three weeks Summer School held in Villard-le-Lans, 1966.
- [17] Venmugil Elango, Norm Rubin, Mahesh Ravishankar, Hariharan Sandanagobalane, and Vinod Grover. Diesel: Dsl for linear algebra and neural net computations on gpus. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, MAPL 2018, pp. 42–51, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450358347. doi: 10.1145/3211346.3211354. URL <https://doi.org/10.1145/3211346.3211354>.
- [18] Daniel Y. Fu, Simran Arora, Jessica Grogan, Isys Johnson, Sabri Eyuboglu, Armin W. Thomas, Benjamin Spector, Michael Poli, Atri Rudra, and Christopher Ré. Monarch mixer: A simple sub-quadratic gemm-based architecture. *37th Conference on Neural Information Processing Systems (NeurIPS 2023)*, 2023.
- [19] Daniel Y. Fu, Tri Dao, Khaled K. Saab, Armin W. Thomas, Atri Rudra, and Christopher Ré. Hungry Hungry Hippos: Towards language modeling with state space models. In *International Conference on Learning Representations*, 2023.
- [20] Daniel Y Fu, Hermann Kumbong, Eric Nguyen, and Christopher Ré. Flashfftconv: Efficient convolutions for long sequences with tensor cores. *arXiv preprint arXiv:2311.05908*, 2023.
- [21] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [22] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *International Conference on Learning Representations (ICLR)*, 2021.
- [23] Ramin Hasani, Mathias Lechner, Tsun-Huang Wang, Makram Chahine, Alexander Amini, and Daniela Rus. Liquid structural state-space models. *arXiv preprint arXiv:2209.12951*, 2022.
- [24] Horace He, Driss Guessous, Yanbo Liang, and Joy Dong. Flexattention: The flexibility of pytorch with the performance of flashattention, 2024. URL <https://github.com/NVIDIA/cutlass>.
- [25] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020.

- [26] Praneeth Kacham, Vahab Mirrokni, and Peilin Zhong. Polysketchformer: Fast transformers via sketching polynomial kernels, 2024. URL <https://arxiv.org/abs/2310.01655>.
- [27] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2020. URL <https://arxiv.org/abs/2006.16236>.
- [28] Feyza Duman Keles, Pruthuvi Mahesakya Wijewardena, and Chinmay Hegde. On the computational complexity of self-attention. In *34th International Conference on Algorithmic Learning Theory*, volume 201, pp. 1–23, 2023.
- [29] NVIDIA. Cuda templates for linear algebra subroutines, 2017. URL <https://github.com/NVIDIA/cutlass>.
- [30] NVIDIA. cuBLAS, 2023. URL <https://docs.nvidia.com/cuda/cublas/>.
- [31] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library, 2019. URL <https://arxiv.org/abs/1912.01703>.
- [32] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Przemyslaw Kazienko, Jan Kocon, and Jiaming et al. Kong. Rwkv: Reinventing rnns for the transformer era. *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.
- [33] Michael Poli, Stefano Massaroli, Eric Nguyen, Daniel Y Fu, Tri Dao, Stephen Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Ré. Hyena hierarchy: Towards larger convolutional language models. *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [34] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *Acm Sigplan Notices*, 48(6):519–530, 2013.
- [35] Nadav Rotem, Jordan Fix, Saleem Abdulrasool, Garret Catron, Summer Deng, Roman Dzhabarov, Nick Gibson, James Hegeman, Meghan Lele, Roman Levenstein, et al. Glow: Graph lowering compiler techniques for neural networks. *arXiv preprint arXiv:1805.00907*, 2018.
- [36] Fabian Schuetze. Reverse-engineering cublas, 2024. URL <https://accu.org/journals/overload/32/181/schuetze/#:~:text=The%20kernels%20provided%20with%20cuBLAS,Ubuntu%2C%20libblas%2C%20ships%20600KB>.
- [37] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision, 2024. URL <https://arxiv.org/abs/2407.08608>.
- [38] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.
- [39] Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, MAPL 2019, pp. 10–19, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367196. doi: 10.1145/3315508.3329973. URL <https://doi.org/10.1145/3315508.3329973>.
- [40] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. Tensor comprehensions: Framework-agnostic high-performance machine learning abstractions. *arXiv preprint arXiv:1802.04730*, 2018.

- [41] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *31st Conference on Neural Information Processing Systems (NIPS 2017)*, 2017.
- [42] A. Vyas, A. Katharopoulos, and F. Fleuret. Fast transformers with clustered attention. In *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2020.
- [43] Richard Wei, Lane Schwartz, and Vikram Adve. Dlvn: A modern compiler infrastructure for deep learning systems. *arXiv preprint arXiv:1711.03016*, 2017.
- [44] Songlin Yang and Yu Zhang. Fla: A triton-based library for hardware-efficient implementations of linear attention mechanism, January 2024. URL <https://github.com/sustcsonglin/flash-linear-attention>.
- [45] Michael Zhang, Simran Arora, Rahul Chalamala, Alan Wu, Benjamin Spector, Aaryan Singhal, Krithik Ramesh, and Christopher Ré. Lolcats: On low-rank linearizing of large language models. 2024. URL <https://arxiv.org/abs/2410.10254>.
- [46] Michael Zhang, Kush Bhatia, Hermann Kumbong, and Christopher Ré. The hedgehog & the porcupine: Expressive linear attentions with softmax mimicry. *International Conference on Learning Representations (ICLR)*, 2024.

Our appendix is organized as follows:

1. Appendix A provides an extended discussion of related work.
2. Appendix B provides a set of kernel listings written in the TK abstractions.
3. Appendix C provides extended discussion of approaches to constructing shared memory layouts and the set of layouts used in TK.

A Related work

We provide an initial discussion of related work in Section 2 and extended discussion here. We discuss frameworks that support AI kernel development and prior work to develop hardware-aware AI algorithms.

CPP embedded libraries Towards raising the level of abstraction and supporting simpler programs, NVIDIA maintains the CuTe and CUTLASS libraries, which are *CUDA* primitives library for graphics, scientific computing, and ML. As discussed in Section 2, both CUTLASS and TK can support the same kernels, since both are C++ embedded libraries. Developers can use the power of the full C++ library, including raw CUDA, when using these frameworks. Distinct from CUTLASS’s objectives, we specifically explore how broad and fast we can go, just using a small number of opinionated abstractions.

Compiler-based libraries Many machine learning frameworks employ high-level computational graph representations for optimizations, such as TVM [10], TensorFlow XLA [1], Glow [35], and DLVM [43]. TVM, for instance, incorporates a flexible tensor expression language and automated schedule optimization. Building upon Halide’s [34] separation of algorithm and schedule, TVM introduces new primitives such as tensorization. These frameworks’ approach differs from THUNDERKITTENS in that they provide a full end-to-end stack that optimizes both graph-level and operator-level transformations, while THUNDERKITTENS concentrates specifically on kernel-level optimizations.

Triton [39] builds on existing approaches to deep learning compilation while introducing novel techniques for efficient tiled computations on GPUs. Recently, tools such as Flex Attention also provide easy to use interfaces to write kernels for attention variants, and compile down to Triton [24]. Unlike XLA and Glow, which use tensor-level IRs and predefined templates, or Tensor Comprehensions [40] and Diesel [17], which rely on polyhedral models, Triton introduces a C-like language (Triton-C) and an LLVM-based IR (Triton-IR) centered around parametric tile variables. This approach supports non-affine tensor indices that polyhedral models struggle with. Triton’s JIT compiler (Triton-JIT) implements tile-level optimizations and an auto-tuner, often enabling high performance on par with hand-tuned libraries. A key difference between TK and Triton is that TK is embedded within CUDA, and as a result its abstractions fail gracefully. In contrast, operations in Triton must be written entirely within its framework.

Hardware-aware AI architectures We are inspired by the success of several prior works that introduce systems-level innovations to improve ML efficiency such as FlashAttention [12, 15, 37] and other optimized attentions [8], FlashFFTConv [20], linear attention kernels [13, 32, 42, 44]. Given the effort required to independently optimize each new architecture, we ask whether a small set of opinionated GPU programming abstractions can obtain kernels for a broad range of AI operations.

Library	Size (Bytes)	Date / Version
CutLASS	22MB	10/22/2024
CuBLAS	689MB	CUDA 12.6
Triton	12.6MB	10/22/2024
TK	<1.0MB	10/22/2024

Table 5: Sizes of various CUDA libraries. For CuTLASS and TK we report the size of the “include/” directory, for CuBLAS we inspect the sizes of the static library files in CUDA, and for Triton we report the combined size of the “include/” directories in Triton and in the core MLIR compiler dependency.

B ThunderKittens kernels

This section first recaps our benchmarking methodology for the results and provides a set of kernels written in the TK LCSF template and tile abstractions:

1. Appendix B.1 GEMM kernel
2. Appendix B.2 Long convolution kernel
3. Appendix B.3 Attention kernel
4. Appendix B.4 Rotary kernel

To introduce the template components, we describe the GEMM kernel in detail in Appendix B.1.

Benchmarking approach Our kernels in Section 4 are benchmarked on an NVIDIA H100 80GB SXM GPU with 10 warmup and 10 timed iterations using timings measured in C++. We also provide Python-bound kernels and benchmarking infrastructure in our repository for reference.

B.1 Matrix multiply

First we show and describe a TK GEMM kernel in the LCSF template.

Each compute warpgroup is responsible for computing $64M$ -row, $64N$ -column chunk of the resulting output matrix. Each compute worker identifies the coordinates for its chunk, zeros its accumulator registers, repeatedly runs large asynchronous matrix multiplies (compute), and finally stores out its tile in the end (finish). The load workers also compute their coordinates, and then repeatedly load chunks of the input matrices (load). Store workers perform asynchronous stores when the compute workers are finished with the chunks (stores).

Tuning the number of workers and pipeline stages The computation is divided into stages, with each stage processing 64 elements along the reduction dimensions of the input matrices. The input pipeline is automatically sized by THUNDERKITTENS if the user does not specify a value. For common configurations of either a (2 compute warpgroup) 128×256 or (3 compute warpgroups) 192×192 output tile per block, it generates a 4-stage pipeline.

Tuning the grid order The greatest complexity of this kernel is in setting the grid parameters. This kernel adopts a 3D stride over the input matrices, which has a significant effect for large matrices which do not fit in L2 cache. The order in which blocks execute strongly influences cache locality and thus available memory bandwidth. To illustrate the magnitude of the effect, comparing the presented scheme versus a naive grid (in which blocks are executed in row-major order) a $4096 \times 4096 \times 4096$ matrix multiply only drops from 767 TFLOPs to 735 TFLOPs, but a $16384 \times 16384 \times 16384$ matrix multiply drops from 797 TFLOPs to 387 TFLOPs, a $> 50\%$ performance degradation.

```
1 using namespace kittens;
2 using namespace kittens::prototype;
3 using namespace kittens::prototype::lcf;
4 template<int M_BLOCK, int N_BLOCK>
5 struct matmul_layout {
6     using base_tile = st_bf<64, 64>;
7     using global_layout = gl<bf16, 1, 1, -1, -1, base_tile>;
8     struct globals { global_layout A, B, C; };
9     struct input_block { base_tile a[M_BLOCK], b[N_BLOCK]; };
10    struct finish_block { base_tile c[M_BLOCK][N_BLOCK]; };
11    struct common_state { int2 coord; };
12    struct consumer_state { rt_fl<16, N_BLOCK*base_tile::cols> accum; };
13 };
```

```

1 template<int _M_BLOCK=2, int _N_BLOCK=4, int _SUPER_M=12>
2 struct matmul_template {
3     static constexpr int M_BLOCK = _M_BLOCK, N_BLOCK = _N_BLOCK, SUPER_M = _SUPER_M;
4     using layout = matmul_layout<M_BLOCK, N_BLOCK>;
5     using wide_tile = st_bf<64, 64*N_BLOCK>;
6     static constexpr int NUM_CONSUMER_WARPS=M_BLOCK*4, INPUT_PIPE_STAGES=4, PRODUCER_BARRIER_ARRIVALS=1;
7     // Helper functions
8     template<bool PERISISTENT_GRID=true> __host__ static inline dim3 grid(int M, int N, int K) {
9         return dim3(PERISISTENT_GRID ? 132 : M*N/(M_BLOCK*N_BLOCK*layout::base_tile::num_elements));
10    }
11    // ThunderKittens template functions
12    __device__ static inline void common_setup(common_setup_args<layout> args) {
13        int Rblocks = args.globals.C.rows / (M_BLOCK*64), Cblocks = args.globals.C.cols / (N_BLOCK*64);
14        int super_rows = (Rblocks/SUPER_M)*SUPER_M,
15            final_rows = Rblocks - super_rows,
16            super_repeat = SUPER_M*Cblocks;
17        int task_id = args.task_iter*gridDim.x + blockIdx.x;
18        if (task_id < super_rows * Cblocks)
19            args.common.coord = { SUPER_M*(task_id/super_repeat) + task_id%SUPER_M,
20                                (task_id%super_repeat)/SUPER_M };
21        else if (task_id < Rblocks*Cblocks) {
22            int remainder_id = task_id - super_rows*Cblocks;
23            args.common.coord = { super_rows + (remainder_id%final_rows), remainder_id/final_rows };
24        }
25        else { // Id is too high, no more work to do
26            args.num_iters = -1;
27            return;
28        }
29        args.num_iters = args.globals.A.cols/64;
30        int id = warpgroup::groupid() == NUM_CONSUMER_WARPS/4 ? 0 : warpgroup::groupid(); // producer sets as
31        ↪ 0
32        args.common.coord = { args.common.coord.x*M_BLOCK + id, args.common.coord.y*N_BLOCK };
33    }
34    struct producer {
35        __device__ static void setup(producer_setup_args<layout> args) {
36            warpgroup::decrease_registers<40>(); // decrease registers for producers
37        }
38        __device__ static void load(producer_load_args<layout> args) {
39            if(warpgroup::warpid() == 0) {
40                tma::expect(args.inputs_arrived, args.input);
41                for(int i = 0; i < M_BLOCK; i++)
42                    tma::load_async(args.input.a[i], args.globals.A,
43                                   {args.common.coord.x+i, args.iter}, args.inputs_arrived);
44                for(int i = 0; i < N_BLOCK; i++)
45                    tma::load_async(args.input.b[i], args.globals.B,
46                                   {args.iter, args.common.coord.y+i}, args.inputs_arrived);
47            }
48        };
49    }
50    struct consumer {
51        __device__ static void setup(consumer_setup_args<layout> args) {
52            warpgroup::increase_registers<232>(); // increase registers for consumers
53            zero(args.state.accum);
54        }
55        __device__ static void compute(consumer_compute_args<layout> args) {
56            warpgroup::mma_AB(
57                args.state.accum, // dest registers
58                args.input.a[warpgroup::groupid()], // A matrix
59                reinterpret_cast<wide_tile&>(args.input.b) // B matrix
60            );
61            warpgroup::mma_async_wait();
62            if(laneid() == 0) arrive(args.inputs_finished);
63        }
64        __device__ static void finish(consumer_finish_args<layout> args) {
65            warpgroup::store(reinterpret_cast<wide_tile&>(args.finish.c[warpgroup::groupid()]), args.state.
66                ↪ accum);
67            warpgroup::sync();
68            if(warpgroup::warpid() == 0) for(int i = 0; i < N_BLOCK; i++) {
69                tma::store_async(args.globals.C, args.finish.c[warpgroup::groupid()][i],
70                                {args.common.coord.x, args.common.coord.y+i});
71                tma::store_async_read_wait(); // wait that store is finished before reusing finish memory
72            }
73            zero(args.state.accum);
74            if(laneid() == 0) arrive(args.finish_finished);
75        }
76    };
77 };

```

Figure 10: Templated matrix multiply kernel which is reasonably competitive with CuBLAS.

B.2 Long convolution

This section shows the long convolution kernel for sequence length 4096, written in the TK abstractions. We use the FFT convolution algorithm, computed via Monarch Matrices, for our long convolution kernel [11, 14, 18].

```
1 using namespace kittens;
2 using namespace kittens::prototype;
3 using namespace kittens::prototype::lcsf;
4 template<int _wg> struct fftconv_4096_layout { // 4096
5     static constexpr int wg = _wg;
6     using seq_tile = st_bf<64, 64>;
7     using seq_layout = gl<bf16, -1, -1, 64, 64, seq_tile>;
8     using filter_layout = cgl<gl<bf16, 1, -1, 64, 64, seq_tile>>;
9     using fft_layout = cgl<gl<bf16, 1, 1, 64, 64>>;
10    struct globals {
11        seq_layout o, x;
12        filter_layout kf;
13        fft_layout f, finv, tw, twinv_t;
14    };
15    struct input_block { seq_tile x[wg]; };
16    struct output_block { seq_tile o[wg]; };
17    struct scratch_block {
18        cst_bf<64, 64> kf, f, finv, tw, twinv_t, tmp[2];
19    };
20    struct consumer_state { int current_head; };
21 };
22 struct fft_4096_template {
23     static constexpr int NUM_CONSUMER_WARPS=8, NUM_CONSUMER_WARPGROUPS=NUM_CONSUMER_WARPS/4, NUM_BLOCKS=1,
24     ↪ OUTPUT_PIPE_STAGES=2, INPUT_PIPE_STAGES=4;
25     using layout = fftconv_4096_layout<NUM_CONSUMER_WARPGROUPS>;
26     // mine
27     __device__ static inline void load_head_data(typename layout::scratch_block &scratch, const layout::
28     ↪ globals &g, int head) {
29         using consumers = group<NUM_CONSUMER_WARPS>;
30         consumers::sync(3);
31         consumers::load(scratch.kf, g.kf, {0, head, 0, 0}); // next chunk
32         consumers::sync(3);
33     }
34     // tk
35     __device__ static void common_setup(common_setup_args<layout> args) {
36         int heads_handled = (args.globals.x.batch+131-blockIdx.x) / 132; // I am guaranteeing batch is
37         ↪ handled by just one block.
38         int iters_per_head = (args.globals.x.batch + NUM_CONSUMER_WARPGROUPS-1) / NUM_CONSUMER_WARPGROUPS;
39         args.num_iters = args.task_iter == 0 ? heads_handled * iters_per_head : -1;
40     }
41     struct producer {
42         __device__ static void setup(producer_setup_args<layout> args) {
43             warpgroup::producer_registers();
44         }
45         __device__ static void load(producer_load_args<layout> args) {
46             int iters_per_head = (args.globals.x.batch + NUM_CONSUMER_WARPGROUPS-1) / NUM_CONSUMER_WARPGROUPS;
47             int head = (args.iter / iters_per_head)*132 + blockIdx.x;
48             int batch = (args.iter % iters_per_head) * NUM_CONSUMER_WARPGROUPS;
49             if(warpgroup::warpid() == args.iter%4) {
50                 tma::expect_bytes(args.inputs_arrived, sizeof(args.input.x[0]) * min((int)NUM_CONSUMER_WARPGROUPS
51                 ↪ , (int)(args.globals.x.batch - batch)));
52                 for(int b = batch; b < batch+NUM_CONSUMER_WARPGROUPS && b < args.globals.x.batch; b++) {
53                     tma::load_async(args.input.x[b-batch], args.globals.x, { b, head, 0, 0 }, args.inputs_arrived);
54                 }
55                 if(laneid() == 0) arrive(args.inputs_arrived, 3); // extra arrivals needed
56                 __syncwarp();
57             }
58         }
59         __device__ static void store(producer_store_args<layout> args) {
60             int iters_per_head = (args.globals.x.batch + NUM_CONSUMER_WARPGROUPS-1) / NUM_CONSUMER_WARPGROUPS;
61             int head = (args.iter / iters_per_head)*132 + blockIdx.x;
62             int batch = (args.iter % iters_per_head) * NUM_CONSUMER_WARPGROUPS;
63             if(warpgroup::warpid() == args.iter%4) {
64                 for(int b = batch; b < batch+NUM_CONSUMER_WARPGROUPS && b < args.globals.x.batch; b++) {
65                     tma::store_async(args.globals.o, args.output.o[b-batch], { b, head, 0, 0 });
66                 }
67                 tma::store_async_read_wait();
68                 if(laneid() == 0) arrive(args.outputs_finished, 4);
69                 __syncwarp();
70             }
71         }
72     };
73 };
74
```

```

1  struct consumer {
2  __device__ static void setup(consumer_setup_args<layout> args) {
3      warpgroup::consumer_registers<NUM_CONSUMER_WARPS/4>();
4      int iters_per_head = (args.globals.x.batch + NUM_CONSUMER_WARPGROUPS-1) / NUM_CONSUMER_WARPGROUPS;
5      args.state.current_head = (0 / iters_per_head)*132 + blockIdx.x; // start for iter 0
6      using consumers = group<NUM_CONSUMER_WARPS>;
7      consumers::load(args.scratch.f,      args.globals.f,      {0, 0, 0, 0});
8      consumers::load(args.scratch.finv,   args.globals.finv,   {0, 0, 0, 0});
9      consumers::load(args.scratch.tw,     args.globals.tw,     {0, 0, 0, 0});
10     consumers::load(args.scratch.twinv_t, args.globals.twinv_t, {0, 0, 0, 0});
11     load_head_data(args.scratch, args.globals, args.state.current_head);
12 }
13 __device__ static void compute(consumer_compute_args<layout> args) {
14
15     int warpgroupid = warpgroup::warpid()/kittens::WARPGROUP_WARPS;
16     int default_barrer_id = warpgroupid + 4;
17
18     // X = F^T X
19     crt_fl<16, 64> mma_reg; // 64 registers
20     crt_bf<16, 64> accum, tmp; // 32 registers each
21     warpgroup::mm_AB(mma_reg.real, args.scratch.f.real, args.input.x[warpgroup::groupid()]);
22     warpgroup::mm_AB(mma_reg.imag, args.scratch.f.imag, args.input.x[warpgroup::groupid()]);
23     warpgroup::mma_async_wait();
24     copy(accum, mma_reg);
25
26     warpgroup::load(tmp, args.scratch.tw); // for twiddle first
27     mul(accum, accum, tmp);
28
29     group<NUM_CONSUMER_WARPS>::sync(2);
30     warpgroup::mm_AB(mma_reg, accum, args.scratch.f);
31     warpgroup::mma_async_wait();
32     copy(accum, mma_reg);
33
34     warpgroup::load(tmp, args.scratch.kf); // for filter second
35     mul(accum, accum, tmp);
36
37     warpgroup::mm_AB(mma_reg, accum, args.scratch.finv);
38     warpgroup::mma_async_wait();
39     copy(accum, mma_reg);
40
41     warpgroup::load(tmp, args.scratch.twinv_t); // twiddle inverse is pre-transposed
42     mul(accum, accum, tmp);
43
44     warpgroup::store(args.scratch.tmp[warpgroup::groupid()], accum); // must store for AtB
45     warpgroup::sync(default_barrer_id);
46
47     warpgroup::mm_AB(mma_reg, args.scratch.finv, args.scratch.tmp[warpgroup::groupid()]); // TODO:
48     ↪ optimize
49     warpgroup::mma_async_wait();
50
51     warpgroup::store(args.output.o[warpgroup::groupid()], mma_reg.real); // COMMENT ME OUT LATER
52     warpgroup::sync(default_barrer_id);
53
54     if(laneid() == 0) {
55         arrive(args.inputs_finished);
56         arrive(args.outputs_arrived);
57     }
58     __syncwarp();
59     int iters_per_head = (args.globals.x.batch + NUM_CONSUMER_WARPGROUPS-1) / NUM_CONSUMER_WARPGROUPS;
60     int next_head = ((args.iter+1) / iters_per_head)*132 + blockIdx.x;
61     if(next_head != args.state.current_head) {
62         load_head_data(args.scratch, args.globals, next_head);
63         args.state.current_head = next_head;
64     }
65 }
66 __device__ static void finish(consumer_finish_args<layout> args) { if(laneid() == 0) arrive(args.
67     ↪ finish_finished); }
68 };

```

Figure 11: A templated convolution kernel for context length 4096, which outperforms FlashFFTConv [20].

B.3 Attention

This section shows non-causal attention at head dimensions 64, 128, in the TK abstractions.

```

1 using namespace kittens;
2 using namespace kittens::prototype;
3 using namespace kittens::prototype::lcf;
4 template<int D, int NUM_WORKERS> struct attn_fwd_layout {
5     using qo_tile = st_bf<64, D>;
6     using kv_tile = st_bf<D==64?192:128, D>;
7     using qo_global = kittens::gl<bf16, -1, -1, -1, D, qo_tile>;
8     using kv_global = kittens::gl<bf16, -1, -1, -1, D, kv_tile>;
9     struct globals { qo_global Q, kv_global K, V; };
10    struct input_block { kv_tile k, v; };
11    struct scratch_block { qo_tile q[NUM_WORKERS]; };
12    struct common_state { int batch, head, seq; };
13    struct consumer_state {
14        rt_fl<16, qo_tile::cols> o_reg;
15        col_vec<rt_fl<16, kv_tile::rows>> max_vec, norm_vec;
16        col_vec<rt_fl<16, kv_tile::rows>> max_vec_last_scaled, max_vec_scaled;
17        rt_fl<16, kv_tile::rows> att_block;
18        rt_bf<16, kv_tile::rows> att_block_mma;
19    };
20 };
21 template<int D> struct attn_fwd_template {
22     static constexpr int NUM_CONSUMER_WARPS = 12, NUM_WORKERS = NUM_CONSUMER_WARPS/4, INPUT_PIPE_STAGES =
23         ↳ 2;
24     using layout = attn_fwd_layout<D, NUM_WORKERS>;
25     __device__ static inline void common_setup(common_setup_args<layout> args) {
26         args.common.batch = blockIdx.z; args.common.head = blockIdx.y; args.common.seq = blockIdx.x;
27         args.num_iters = args.task_iter == 0 ? args.globals.K.rows/layout::kv_tile::rows : -1;
28     }
29     struct producer {
30         __device__ static inline void setup(producer_setup_args<layout> args) {
31             warpgroup::producer_registers();
32         }
33         __device__ static inline void load(producer_load_args<layout> args) {
34             if(warpgroup::warpid() == 0) {
35                 tma::expect(args.inputs_arrived, args.input);
36                 tma::load_async(args.input.k, args.globals.K, {args.common.batch, args.common.head, args.iter,
37                     ↳ 0}, args.inputs_arrived);
38                 tma::load_async(args.input.v, args.globals.V, {args.common.batch, args.common.head, args.iter,
39                     ↳ 0}, args.inputs_arrived);
40             }
41             else if(laneid() == 0) arrive(args.inputs_arrived);
42         }
43     };
44     struct consumer {
45         __device__ static inline void setup(consumer_setup_args<layout> args) {
46             warpgroup::consumer_registers<NUM_WORKERS>();
47             if((args.common.seq*NUM_WORKERS + warpgroup::groupid())*layout::qo_tile::rows < args.globals.Q.rows
48                 ↳ ) // out of bounds?
49                 warpgroup::load(args.scratch.q[warpgroup::groupid()], args.globals.Q,
50                     {args.common.batch, args.common.head, args.common.seq*NUM_WORKERS+warpgroup::
51                     ↳ groupid(), 0});
52             zero(args.state.o_reg);
53             zero(args.state.norm_vec);
54             neg_infty(args.state.max_vec);
55             warpgroup::sync(warpgroup::groupid());
56         }
57         __device__ static inline void compute(consumer_compute_args<layout> args) {
58             constexpr float TEMPERATURE_SCALE = (D == 128) ? 0.08838834764f*1.44269504089f : 0.125f
59                 ↳ *1.44269504089f;
60             // A = Q @ K.T
61             warpgroup::mm_ABt(args.state.att_block, args.scratch.q[warpgroup::groupid()], args.input.k);
62             mul(args.state.max_vec_last_scaled, args.state.max_vec, TEMPERATURE_SCALE);
63             warpgroup::mma_async_wait();

```



```

1      // softmax
2      row_max(args.state.max_vec, args.state.att_block, args.state.max_vec); // accumulate onto the
3      ↪ max_vec
4      mul(args.state.max_vec_scaled, args.state.max_vec, TEMPERATURE_SCALE);
5      mul(args.state.att_block, args.state.att_block, TEMPERATURE_SCALE);
6      sub_row(args.state.att_block, args.state.att_block, args.state.max_vec_scaled);
7      exp2(args.state.att_block, args.state.att_block);
8      sub(args.state.max_vec_last_scaled, args.state.max_vec_last_scaled, args.state.max_vec_scaled);
9      exp2(args.state.max_vec_last_scaled, args.state.max_vec_last_scaled);
10     mul(args.state.norm_vec, args.state.norm_vec, args.state.max_vec_last_scaled);
11     row_sum(args.state.norm_vec, args.state.att_block, args.state.norm_vec); // accumulate onto the
12     ↪ norm_vec
13     mul_row(args.state.o_reg, args.state.o_reg, args.state.max_vec_last_scaled); // normalize o_reg
14     ↪ before mma
15     copy(args.state.att_block_mma, args.state.att_block); // convert to bf16 for mma
16     // 0 += A @ V
17     warpgroup::mma_AB(args.state.o_reg, args.state.att_block_mma, args.input.v);
18     warpgroup::mma_async_wait();
19     if(laneid() == 0) arrive(args.inputs_finished); // done!
20 }
21 __device__ static inline void finish(consumer_finish_args<layout> args) {
22     if((args.common.seq*NUM_WORKERS+warpgroup::groupid())*64 >= args.globals.Q.rows) return; // out of
23     ↪ bounds?
24     div_row(args.state.o_reg, args.state.o_reg, args.state.norm_vec);
25     auto &o_smem = reinterpret_cast<typename> layout::qo_tile&>(args.scratch.q[warpgroup::groupid()]);
26     warpgroup::store(o_smem, args.state.o_reg);
27     warpgroup::sync(warpgroup::groupid());
28     if(warpgroup::warpid() == 0)
29         tma::store_async(args.globals.0, o_smem, {args.common.batch, args.common.head, args.common.seq*
30         ↪ NUM_WORKERS+warpgroup::groupid(), 0});
31 }
32 };
33 // kernel is kittens::prototype::lcf::kernel<attn_fwd_template<HEAD_DIM>>;

```

Figure 12: A templated non-causal attention kernel for head dims. 64 and 128 that competes with FlashAttention-3.

B.4 Rotary positional encodings

This section shows the rotary kernel for head dimension 128, written in the TK abstractions.

```
1 using namespace kittens;
2 using namespace kittens::prototype;
3 using namespace kittens::prototype::lcsf;
4 template<int _headdim, int _warps> struct rotary_layout {
5     static constexpr int headdim = _headdim, warps = _warps;
6     using seq_tile = st_bf<16, headdim>;
7     using seq_global = gl<bf16, -1, -1, -1, headdim, seq_tile>;
8     using rope_global = gl<bf16, 1, 1, -1, headdim/2>;
9     struct globals {
10         seq_global o, x;
11         rope_global sin, cos;
12         int batches; // how many batches per block, for sizing grid
13     };
14     struct input_block { seq_tile x[warps]; };
15     struct output_block { seq_tile o[warps]; };
16     struct producer_state { int active_warps; };
17     struct consumer_state { rt_fl<16, headdim/2> sin, cos; }; // long-resident tiles
18 };
19 template<int _headdim> struct rotary_template {
20     static constexpr int headdim=_headdim, NUM_CONSUMER_WARPS=8, NUM_BLOCKS=1, OUTPUT_PIPE_STAGES=3,
21     ↪ INPUT_PIPE_STAGES=3;
22     using layout = rotary_layout<headdim, NUM_CONSUMER_WARPS>;
23     __device__ static inline void common_setup(common_setup_args<layout> args) {
24         if(args.task.iter == 0) {
25             args.num_iters = min(args.globals.batches, (int)(args.globals.x.batch-blockIdx.y*args.globals.
26             ↪ batches)) * args.globals.x.depth; // batches*heads handled by block
27         }
28         else args.num_iters = -1;
29     }
30     struct producer {
31         __device__ static void setup(producer_setup_args<layout> args) {
32             warpgroup::producer_registers();
33             args.state.active_warps = min((int)NUM_CONSUMER_WARPS,
34             ↪ (int)(args.globals.x.rows/16 - blockIdx.x*NUM_CONSUMER_WARPS));
35         }
36         __device__ static void load(producer_load_args<layout> args) {
37             if(warpgroup::warpid() == args.iter%4) {
38                 kittens::coord idx = { blockIdx.y*args.globals.batches+args.iter/args.globals.x.depth,
39                 ↪ args.iter%args.globals.x.depth,
40                 ↪ blockIdx.x*NUM_CONSUMER_WARPS,
41                 ↪ 0 };
42                 tma::expect_bytes(args.inputs_arrived, sizeof(layout::seq_tile)*args.state.active_warps);
43                 for(int i = 0; i < args.state.active_warps; i++) {
44                     tma::load_async(args.input.x[i], args.globals.x, {idx.b,idx.d,idx.r+i,idx.c}, args.
45                     ↪ inputs_arrived);
46                 }
47                 if(laneid() == 0) arrive(args.inputs_arrived, 3);
48                 __syncwarp();
49             }
50         }
51         __device__ static void store(producer_store_args<layout> args) {
52             if(warpgroup::warpid() == args.iter%4) {
53                 kittens::coord idx = { blockIdx.y*args.globals.batches+args.iter/args.globals.x.depth,
54                 ↪ args.iter%args.globals.x.depth,
55                 ↪ blockIdx.x*NUM_CONSUMER_WARPS,
56                 ↪ 0 };
57                 for(int i = 0; i < args.state.active_warps; i++) {
58                     tma::store_async(args.globals.o, args.output.o[i], {idx.b,idx.d,idx.r+i,idx.c});
59                 }
60                 tma::store_async_read_wait();
61                 if(laneid() == 0) arrive(args.outputs_finished, 4);
62                 __syncwarp();
63             }
64         }
65     };
66 };
67 }
```

```

1  struct consumer {
2  __device__ static void setup(consumer_setup_args<layout> args) {
3      warpgroup::consumer_registers<NUM_CONSUMER_WARPS/4>();
4      kittens::coord idx = { blockIdx.x*NUM_CONSUMER_WARPS + warpid(), 0 };
5      load(args.state.sin, args.globals.sin, idx); // could be better coalesced but doing just once
6      load(args.state.cos, args.globals.cos, idx);
7  }
8  __device__ static void compute(consumer_compute_args<layout> args) {
9      rt_fl<16, headdim> x;
10     rt_fl<16, headdim/2> x1, x2, temp1, temp2;
11     load(x, args.input.x[warpid()]);
12     if(laneid() == 0) arrive(args.inputs_finished);
13     __syncwarp();
14     for(int i = 0; i < headdim/32; i++) {
15         #pragma unroll
16         for(int j = 0; j < 4; j++) {
17             x1.tiles[0][i].data[j] = x.tiles[0][i].data[j];
18             x2.tiles[0][i].data[j] = x.tiles[0][i+headdim/32].data[j];
19         }
20     }
21     mul(temp1, x1, args.state.cos);
22     mul(temp2, x2, args.state.cos);
23     mul(x2, x2, -1.f);
24     mul(x1, x1, args.state.sin);
25     mul(x2, x2, args.state.sin);
26     add(temp1, temp1, x2);
27     add(temp2, temp2, x1);
28     for(int i = 0; i < headdim/32; i++) {
29         #pragma unroll
30         for(int j = 0; j < 4; j++) {
31             x.tiles[0][i].data[j] = temp1.tiles[0][i].data[j];
32             x.tiles[0][i+headdim/32].data[j] = temp2.tiles[0][i].data[j];
33         }
34     }
35     store(args.output.o[warpid()], x);
36     __syncwarp();
37     if(laneid() == 0) arrive(args.outputs_arrived);
38 }
39 __device__ static void finish(consumer_finish_args<layout> args) {
40     if(laneid() == 0) arrive(args.finish_finished); // nothing to do here
41 }
42 };
43 };

```

Figure 13: A templated rotary kernel for head dim. 128 that outperforms popular Triton baselines.

C Shared memory layouts

To illustrate some of the choices available in shared memory layouts, this appendix outlines six different shared memory layouts for GPU tiles: a naive row-major layout, a padded layout, a simple swizzled layout, and three more specialized swizzled layouts. We are particularly interested in which memory *banks* (numbered from 00 to 31) store each element of the tile; for each layout, we color and label the element of the tile accordingly. We illustrate all layouts using a 32×64 16-bit tile.

C.1 Naive layout

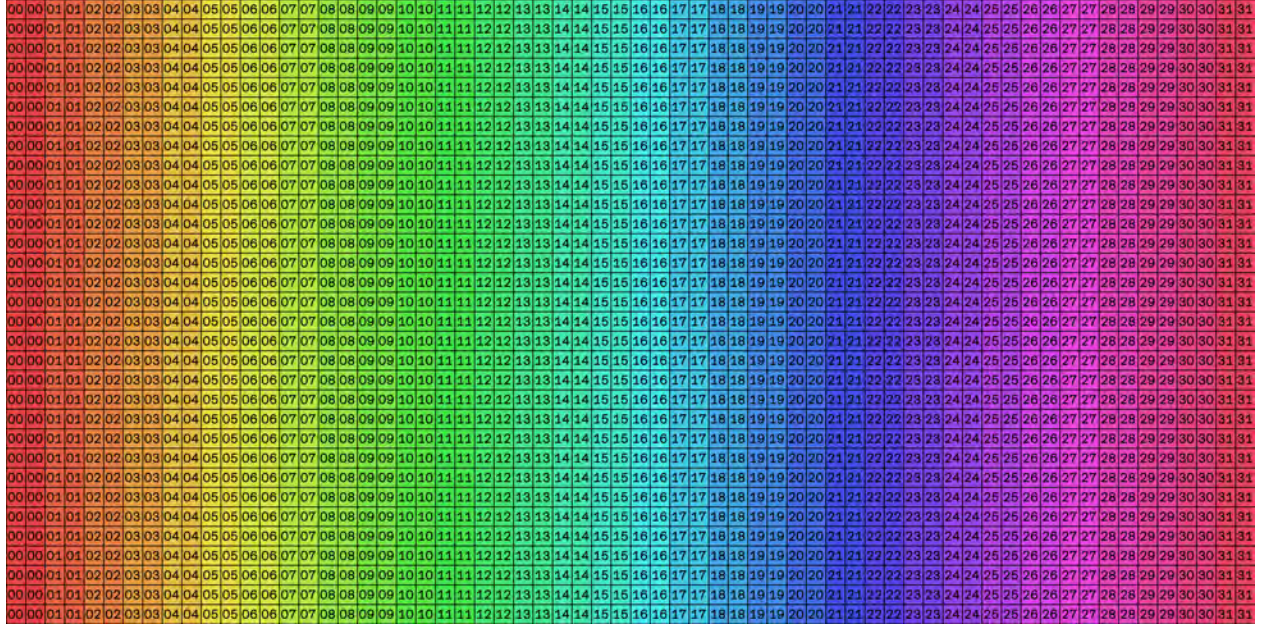


Figure 14: Row-major shared memory layout.

A row-major layout, illustrated in figure 14, is among the simplest layouts. It has the benefit of accurately reflecting tensor layouts in HBM. Furthermore, for access patterns that access row-wise, it has no bank conflicts. But when loading or storing tensor core register layouts, it suffers 8-way bank conflicts, and is thus extremely slow.

```

1 bf16* naive_layout(bf16 *data, int r, int c) {
2     return &data[r * columns + c];
3 }

```


C.2 Padded layout

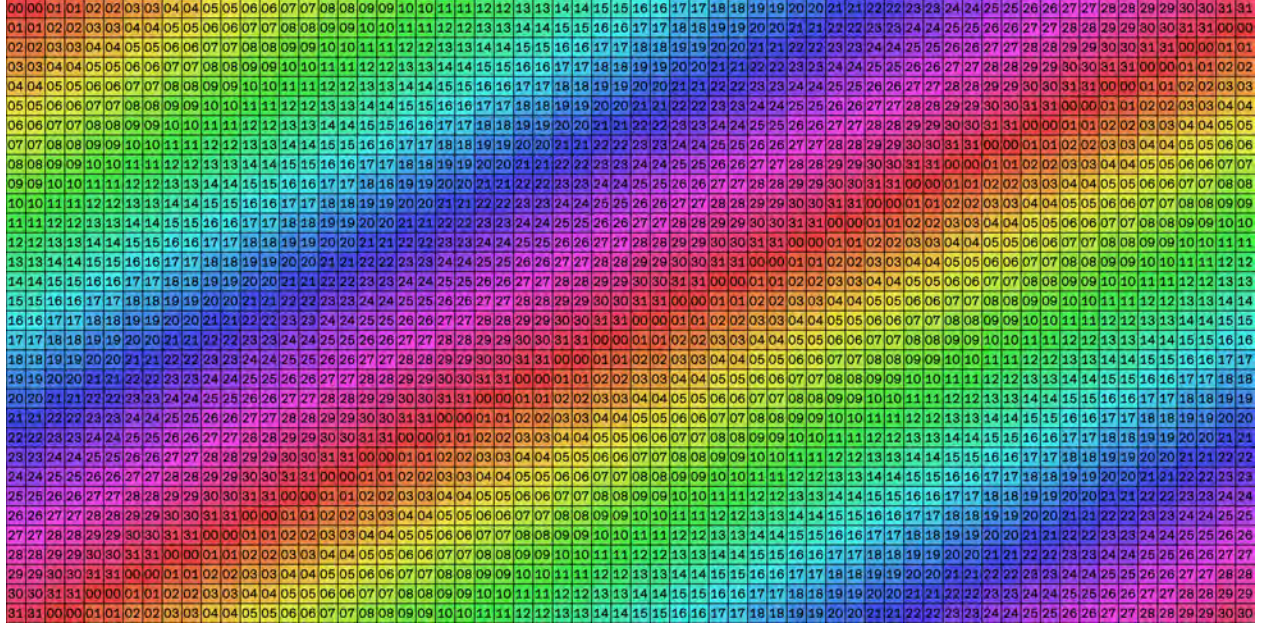


Figure 15: Padded shared memory layout.

A common solution to these bank conflicts is to “pad” each row by one memory bank, thereby introducing an offset to shift consecutive elements of a column into different memory banks. This eliminates bank conflicts, but creates misaligned addresses which interferes with fast instructions that require aligned addresses.

```

1  bf16* padded_layout(bf16 *data, int r, int c) {
2      return &data[r * (columns+1) + c];
3  }

```


C.3 Naive Swizzled Layout

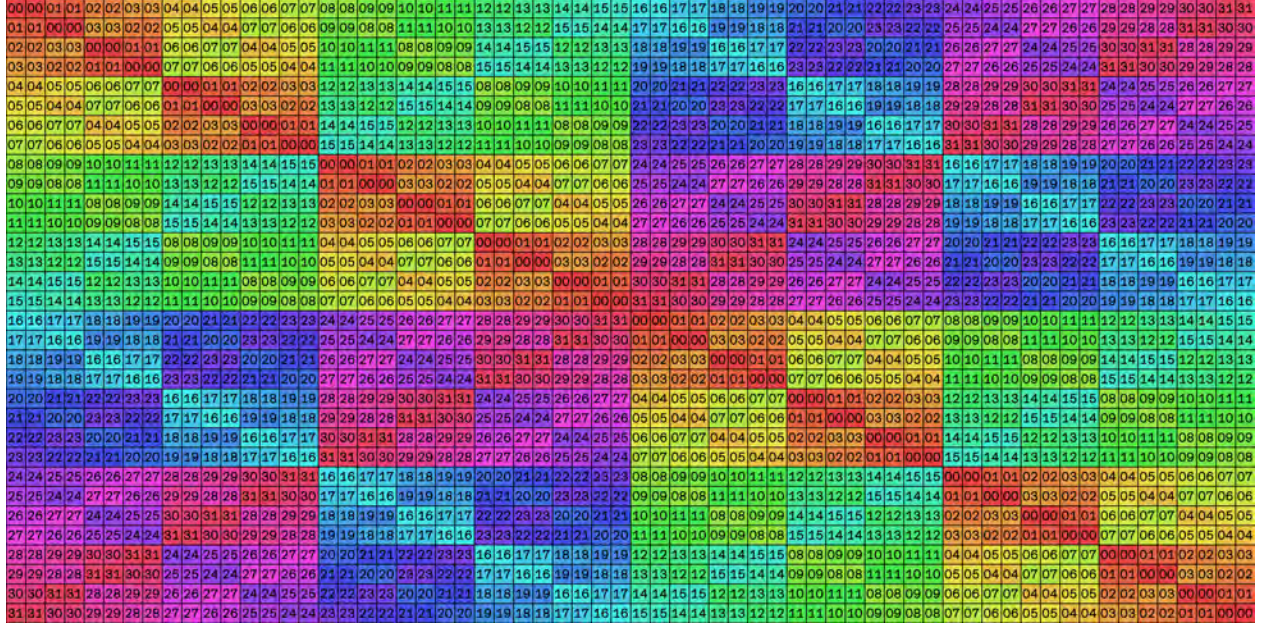


Figure 16: Naive swizzled shared memory layout.

A third option is to “swizzle” the memory, in which progressive rows are reshuffled to alter their banking. This layout accomplishes this by xor’ing the index with the row, which eliminates bank conflicts and has aligned addresses. However, this layout lacks hardware support for [HGMMA](#) and [UTMA](#) instructions, which are particularly important on H100 GPUs for achieving high performance. We illustrate a simple swizzling pattern here:

```
1  bf16* row_swizzled_layout(bf16 *data, int r, int c) {
2      uint64_t addr = (uint64_t)&data[r * columns + c];
3      return (bf16*)(addr ^ (r << 2));
4  }
```

C.4 32 byte swizzling

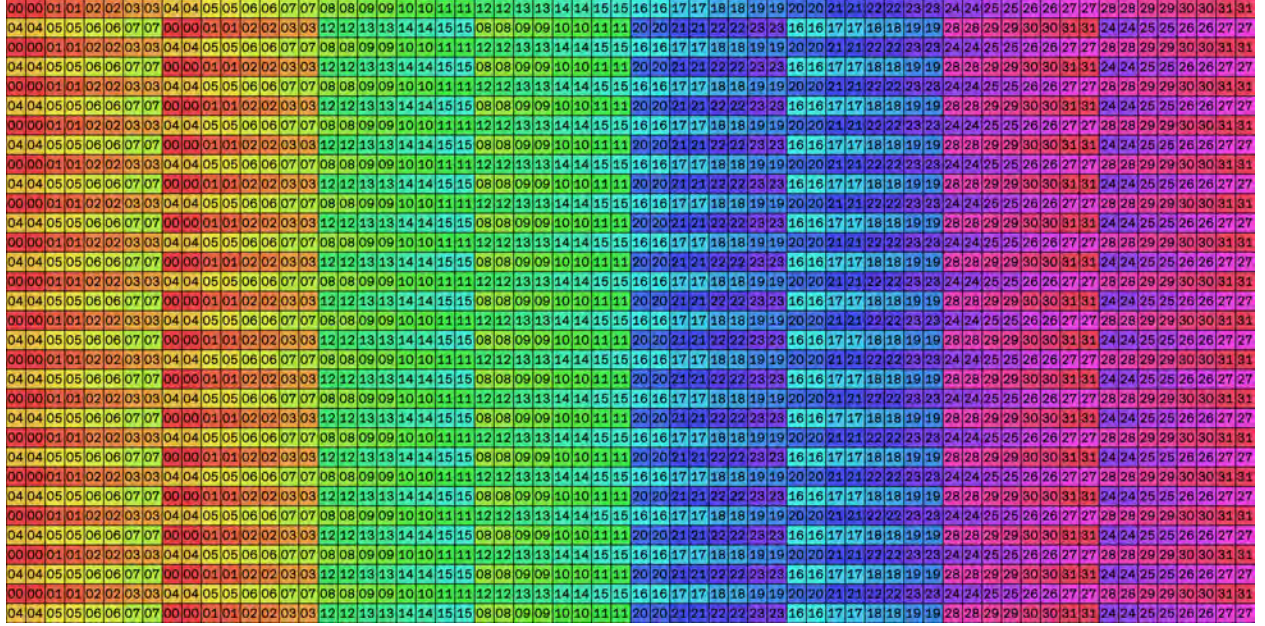


Figure 17: 32 byte swizzled shared memory layout.

32 byte swizzling is the first of a family of layouts (of which we will examine three), where instead of swizzling the index with the row, the memory address is instead swizzled directly with itself. This layout is defined by the following C code:

```
1 bf16* swizzled_layout_32B(bf16 *data, int r, int c) {
2     uint64_t addr = (uint64_t)&data[r * columns + c];
3     return (bf16*)(addr ^ (((addr % (32*8)) >> 7) << 4));
4 }
```

This layout suffers from 4-way bank conflicts, but is valid for all tiles whose width is a multiple of 16. However, importantly, it has (as do its siblings below) hardware support from both [HGMMA](#) and [UTMA](#) instructions.

C.5 64 byte swizzling



Figure 18: 64 byte swizzled shared memory layout.

64 byte swizzling is a layout similar to 32 byte swizzling with a more aggressive pattern:

```

1  bf16* swizzled_layout_64B(bf16 *data, int r, int c) {
2      uint64_t addr = (uint64_t)&data[r * columns + c];
3      return (bf16*)(addr ^ (((addr % (64*8)) >> 7) << 4));
4  }

```

64 byte swizzling suffers from just 2-way bank conflicts, but is only valid for tiles whose width is a multiple of 32 (for half-precision types, or 16 for full-precision).

C.6 128 byte swizzling.



Figure 19: 128 byte swizzled shared memory layout.

128 byte swizzling is a further extension of its kin:

```

1 bfi16* swizzled_layout_128B(bfi16 *data, int r, int c) {
2     uint64_t addr = (uint64_t)&data[r * columns + c];
3     return (bfi16*)(addr ^ (((addr % (128*8)) >> 7) << 4));
4 }

```

Finally, 128 byte swizzling has no bank conflicts, but is only valid for half-precision tiles whose width is a multiple of 64.

C.7 ThunderKittens

After substantial evaluation of these layouts, we concluded that the three final layouts were the three most important, because **HGMMMA** and **UTMA** instructions are critical to high performance, and furthermore that they are good enough to yield high performance across many kernels. Correspondingly, depending on the width of the tile at compile time we select the highest level of swizzling possible to minimize bank conflicts.